

MACROFIT AI

Resumen del quinto bloque de trabajo

DIA 5 | PostgreSQL, SQLAlchemy y primera persistencia real

Autor	Matias Gonzalez
Fecha	4 de agosto de 2026
Rama	feature/backend-setup
Commit principal	9dc2161 - feat: configure PostgreSQL persistence

Objetivo del documento

Registrar la instalacion y configuracion de PostgreSQL, la integracion con SQLAlchemy, la proteccion de credenciales, la creacion de los modelos User y Profile y la verificacion de las primeras tablas reales de MacroFit AI.

1. Hito alcanzado

MacroFit AI ha pasado de usar datos simulados a disponer de una infraestructura de persistencia real. El backend ya puede conectarse a PostgreSQL mediante SQLAlchemy y cuenta con las tablas users y profiles creadas fisicamente.

FastAPI -> SQLAlchemy -> psycopg2 -> PostgreSQL

Tecnologias verificadas

Componente	Version / estado	Funcion
PostgreSQL	18.4	Servidor de base de datos relacional.
SQLAlchemy	2.0.51	ORM entre objetos Python y tablas SQL.
psycopg2-binary	2.9.12	Driver de conexion con PostgreSQL.
pydantic-settings	2.14.2	Carga segura de configuracion desde .env.

Base de datos y usuario

- Base de datos creada: macrofit_ai.
- Usuario de aplicacion creado: macrofit_app.
- El backend no utiliza el superusuario postgres.
- El usuario macrofit_app dispone de permisos sobre el esquema public.

2. Configuracion segura

Las credenciales no se almacenan en el codigo fuente. Se guardan en backend/.env, archivo ignorado por Git, mientras que .env.example documenta las variables necesarias sin exponer secretos.

Variables configuradas

Variable	Proposito
DB_HOST	Servidor PostgreSQL, actualmente localhost.
DB_PORT	Puerto del servidor, 5432.
DB_NAME	Nombre de la base macrofit_ai.
DB_USER	Usuario limitado macrofit_app.
DB_PASSWORD	Contraseña privada, solo presente en .env.

- Git confirma que .env esta ignorado mediante la regla .gitignore:6:.env.
- .env.example se sube al repositorio para facilitar la configuracion local.
- La URL se construye con SQLAlchemy URL.create(), evitando errores con caracteres especiales en la contraseña.
- pool_pre_ping=True verifica las conexiones antes de reutilizarlas.

Infraestructura creada

Archivo	Responsabilidad
app/db/base.py	Base declarativa de los modelos ORM.
app/db/database.py	Settings, engine, SessionLocal y get_db().
app/db/init_db.py	Registro de modelos y creacion inicial de tablas.
app/models/user.py	Modelo ORM de users.
app/models/profile.py	Modelo ORM de profiles.

3. Tablas reales creadas

El comando `python -m app.db.init_db` ejecuto `Base.metadata.create_all(bind=engine)`. PostgreSQL creo correctamente las tablas `users` y `profiles` bajo el propietario `macrofit_app`.

Tabla users

Columna	Tipo / restriccion
id	INTEGER, clave primaria, autoincremental.
name	VARCHAR(100), obligatorio.
email	VARCHAR(255), obligatorio y unico.
password_hash	VARCHAR(255), obligatorio.
preferred_language	VARCHAR(10), obligatorio.
is_active	BOOLEAN, obligatorio.
created_at / updated_at	TIMESTAMP, obligatorios.

Tabla profiles

Columna	Tipo / restriccion
id	INTEGER, clave primaria, autoincremental.
user_id	INTEGER, unico y FK hacia users.id.
birth_date	DATE, obligatorio.
biological_sex	VARCHAR(20), obligatorio.
height_cm / current_weight_kg	DOUBLE PRECISION, obligatorios.
goal / activity_level	VARCHAR(50), obligatorios.

La combinacion de FOREIGN KEY y UNIQUE en `profiles.user_id` implementa una relacion uno a uno: cada usuario puede tener como maximo un perfil.

4. Errores resueltos y aprendizajes

Error	Causa y solucion
ModuleNotFoundError: pydantic_settings	Se estaba usando Python global. Se activo backend/.venv.
Host incorrecto en psycopg2	La contraseña contenia caracteres especiales. Se uso URL.create() y se cambio la contraseña expuesta.
No module named backend	Desde backend el paquete raiz es app. Se corrigieron imports a from app...
psql no reconocido	PostgreSQL no estaba en PATH. Se añadio temporalmente su carpeta bin.
Ruta escrita como comando	Para cambiar de carpeta se debe usar cd antes de la ruta.

Versionado

- Commit creado: 9dc2161 feat: configure PostgreSQL persistence.
- Rama sincronizada: feature/backend-setup.
- El estado final de Git fue working tree clean.
- La base de datos no se sube a GitHub; se sube el codigo necesario para reconstruirla.

Proximos pasos

- Crear un usuario real desde POST /auth/register.
- Aplicar hash seguro a la contraseña antes de guardarla.
- Consultar el usuario mediante UserRepository.
- Implementar login real y tokens JWT.
- Introducir Alembic para migraciones futuras.
- Añadir pruebas automatizadas de persistencia y autenticacion.